C:\hpc-23-feb-2026\.\15\eg132.c

```
// never ever do this as done in this example
#include<stdio.h>
int main()
{
void *p;
int x=50;
int j;
p=(void *)x;
j=(int)p;
printf("%d\n",j);
return 0;
}
```

C:\hpc-23-feb-2026\.\15\eg133.c

```
// never ever do this as done in this example
#include<stdio.h>
#include<stdlib.h>
int main()
{
int x=50;
int *p;
p=(int *)malloc(sizeof(int));
*p=x;
// the upper part is somewhere else (in some other function) (return type is void *
void *k=(void *)p;
int *i=(int *)k;
int j=*i;
printf("%d\n",j);
free(i);
return 0;
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\15\eg134.c

```
#include<stdio.h>
#include<pthread.h>
void * add(void *ptr)
{
int *k=(int *)ptr;
int sum=k[0]+k[1];
pthread_exit((void *)sum); // should not done in this way
return nullptr;
}
int main()
{
void *hh;
pthread_t pid;
int x[2]={10,20};
int result;
int i=pthread_create(&pid,nullptr,add,x);
// value of i may be equal to EAGAIN, EPERM, ENOMEM,  EINVAL
if(i)
{
printf("Unable to create thread\n");
return 1;
}
pthread_join(pid,&hh);
result=(int)hh;
printf("Result : %d\n",result);
return 0;
}
```

C:\hpc-23-feb-2026\.\15\eg135.c

```
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
void * add(void *ptr)
{
int *k=(int *)ptr;
int *p;
p=(int *)malloc(sizeof(int));
*p=k[0]+k[1];
pthread_exit((void *)p);
return nullptr;
}
int main()
{
int *result;
pthread_t pid;
int x[2]={10,20};
int i=pthread_create(&pid,nullptr,add,x);
// value of i may be equal to EAGAIN, EPERM, ENOMEM,  EINVAL
if(i)
{
printf("Unable to create thread\n");
return 1;
}
pthread_join(pid,(void **)&result);
printf("Result : %d\n",*result);
free(result);
return 0;
}
```

C:\hpc-23-feb-2026\.\15\eg136.c

```
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
void * add(void *ptr)
{
int *k=(int *)ptr;
k[2]=k[0]+k[1];
pthread_exit(NULL);
return nullptr;
}
int main()
{
int *result;
pthread_t pid;
int x[3]={10,20,0};
int i=pthread_create(&pid,nullptr,add,x);
// value of i may be equal to EAGAIN, EPERM, ENOMEM,  EINVAL
if(i)
{
printf("Unable to create thread\n");
return 1;
}
pthread_join(pid,NULL);
printf("Result : %d\n",x[2]);
return 0;
}
```

C:\hpc-23-feb-2026\.\15\eg137.c

```
// eg135.c uses pthread_exit((void *)p)
// eg137.c does not use the technique used in eg135.c instead the function
//    running on separate thread returns.
// No difference is seen
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
void * add(void *ptr)
{
int *k=(int *)ptr;
int *p;
p=(int *)malloc(sizeof(int));
*p=k[0]+k[1];
return (void *)p;
}
int main()
{
int *result;
pthread_t pid;
int x[2]={10,20};
int i=pthread_create(&pid,nullptr,add,x);
// value of i may be equal to EAGAIN, EPERM, ENOMEM,  EINVAL
if(i)
{
printf("Unable to create thread\n");
return 1;
}
pthread_join(pid,(void **)&result);
printf("Result : %d\n",*result);
free(result);
return 0;
}
```

C:\hpc-23-feb-2026\.\15\eg138.c

```
// this application will fail if denominator is zero
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
int divide(int numerator,int denominator)
{
return numerator/denominator;
}
void * do_something(void *ptr)
{
int *p;
int *m;
p=(int *)ptr;
m=(int *)malloc(sizeof(int));
*m=divide(p[0],p[1]);
pthread_exit((void *)m);
return NULL;
}
int main()
{
int e,f;
int x[2];
int *g;
pthread_t pid;
int i;
printf("Enter a number : ");
scanf("%d",&e);
printf("Enter another number : ");
scanf("%d",&f);
x[0]=e;
x[1]=f;
i=pthread_create(&pid,NULL,do_something,(void *)x);
if(i)
{
printf("Unable to created threads\n");
return 0;
}
pthread_join(pid,(void **)&g);
if(g==NULL)
{
printf("Unable to divide\n");
}
else
{
printf("Result : %d\n",*g);
free(g);
}
return 0;
```

```
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\15\eg139.c

```
// issue related to eg138 resolved in eg139
// but a new issue has arised related to memory leak
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
int divide(int numerator,int denominator)
{
if(denominator==0) pthread_exit(NULL);
return numerator/denominator;
}
void * do_something(void *ptr)
{
int *p;
int *m;
p=(int *)ptr;
m=(int *)malloc(sizeof(int));
*m=divide(p[0],p[1]);
pthread_exit((void *)m);
return NULL;
}
int main()
{
int e,f;
int x[2];
int *g;
pthread_t pid;
int i;
printf("Enter a number : ");
scanf("%d",&e);
printf("Enter another number : ");
scanf("%d",&f);
x[0]=e;
x[1]=f;
i=pthread_create(&pid,NULL,do_something,(void *)x);
if(i)
{
printf("Unable to created threads\n");
return 0;
}
pthread_join(pid,(void **)&g);
if(g==NULL)
{
printf("Unable to divide\n");
}
else
{
printf("Result : %d\n",*g);
free(g);
```

```
}
return 0;
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\15\eg140.cpp

```
// winpthread api has a bug in case of pthread_exit
// does not unwind and destruct local object/variables
#include<iostream>
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
using namespace std;
class aaa
{
public:
aaa()
{
cout<<"aaa Constructor"<<endl;
}
~aaa()
{
cout<<"aaa Destructor"<<endl;
}
};
class bbb
{
public:
bbb()
{
cout<<"bbb Constructor"<<endl;
}
~bbb()
{
cout<<"bbb Destructor"<<endl;
}
};
int divide(int numerator,int denominator)
{
aaa a;
if(denominator==0) pthread_exit(NULL);
return numerator/denominator;
}
void * do_something(void *ptr)
{
bbb b;
int *p;
int *m;
p=(int *)ptr;
m=(int *)malloc(sizeof(int));
*m=divide(p[0],p[1]);
pthread_exit((void *)m);
return NULL;
}
```

```
int main()
{
int e,f;
int x[2];
int *g;
pthread_t pid;
int i;
printf("Enter a number : ");
scanf("%d",&e);
printf("Enter another number : ");
scanf("%d",&f);
x[0]=e;
x[1]=f;
i=pthread_create(&pid,NULL,do_something,(void *)x);
if(i)
{
printf("Unable to created threads\n");
return 0;
}
pthread_join(pid,(void **)&g);
if(g==NULL)
{
printf("Unable to divide\n");
}
else
{
printf("Result : %d\n",*g);
free(g);
}
return 0;
}
```